

# InCommon Frontend Agentic Engineer - What to Learn

## Overview

InCommon's Frontend Agentic Engineer role is about extending existing products with AI-native capabilities (MCP servers, agentic workflows) while working full-stack. Your React/React Native/Next.js background covers most of the frontend ask directly. The gaps below are mostly about the *production, standardized* version of things you already do informally (agentic coding, tool integration).

## MUST learn

### MCP (Model Context Protocol) — production use, not just as a Claude Code user

- What it is: an open protocol (from Anthropic) that lets an LLM/agent call external tools and data sources through a standard client-server interface — think of it as “USB-C for AI agents.” You already use Claude Code (which is an MCP client) daily, so you know the consumer side.
- Why the JD wants it: the role explicitly asks for building “MCP servers for agent integration” — i.e. you'll be the one *exposing* your product's data/actions as MCP tools, not just consuming them.
- The 20% to learn: how to write a minimal MCP server (Node/TypeScript SDK: `@modelcontextprotocol/sdk`) that exposes a few tools/resources, and how a client (like Claude Desktop or Cursor) discovers and calls them. This maps almost exactly onto the Zod-typed tool-calling you already built in Echo (3 tools: search, escalate, resolve) — MCP is just a standardized transport/schema layer around that same idea.
- Bridge: you already architected an agent that calls typed tools (Echo). MCP servers are the same pattern, wrapped in a spec so ANY agent client (not just your own backend) can call them.

## Angular

- What it is: a full opinionated frontend framework (unlike React's library-first approach) — built-in DI, RxJS-based reactivity, TypeScript-first, CLI-driven project structure.
- Why the JD wants it: listed as one of several acceptable frameworks (Node/TypeScript, React, **Angular**, Python) — likely because some legacy/established products at InCommon use it.
- The 20% to learn: components + templates (similar mental model to JSX, just different syntax), services + dependency injection (the main conceptual jump from React), and basic RxJS observables (compare mentally to your Redux/async-thunk patterns). You don't need NgRx or advanced RxJS operators for an interview-level grasp.
- Bridge: your React component/props/state model transfers almost directly to Angular's component/@Input/@Output model; the new part is DI and Observables replacing plain props/callbacks.

## GOOD to learn

### AI-SDLC workflows & testing protocols at scale

- What it is: formalizing how AI coding tools fit into the software development lifecycle — e.g. agent-generated PRs still going through code review, automated test generation/verification loops, guardrails for AI-written code before merge.
- Why the JD wants it: “establish AI-SDLC workflows and testing protocols while maintaining quality at scale” is a named responsibility — they want someone who can define process, not just use the tools.
- The 20%: skim how teams structure AI-assisted PR review (e.g. requiring a human-reviewed diff summary, running existing test suites against agent-written code, treating agent output as “untrusted until reviewed” the same way you already treat AI-generated code in Vibe's sandboxed execution).

## GitHub Copilot

- What it is: inline AI code-completion in the editor (contrast with Cursor/Claude Code's more

agentic, multi-file, chat-driven workflow you already use).

- Why the JD wants it: named alongside Cursor and Claude Code as an example agentic coding tool — you likely won't be tested deeply on it, just expected to be comfortable switching tools.
- The 20%: install it in VS Code once, note the difference (Copilot = autocomplete-style suggestions; Cursor/Claude Code = full agentic edits across files) so you can speak to the distinction in an interview.

### **Not worth deep prep**

Enterprise stack (Java, .NET, MSSQL, Oracle) and financial/trading-market experience are listed as “standout,” not required — your .NET exposure (already on your resume) and general SQL/RDBMS knowledge are enough to speak to intelligently if asked; no need to learn Oracle or a trading domain from scratch.